

100 EXERCÍCIOS DE VETORES EM C

Do básico ao difícil, em quatro níveis · com gabarito comentado

São 100 exercícios que usam **apenas vetores unidimensionais**. Nada de matrizes, structs, listas encadeadas ou arquivos: o objetivo é dominar completamente a estrutura mais básica antes de seguir adiante. A dificuldade sobe de forma deliberada:

Nível	Números	O que exige
Fácil	1 a 40	ler, mostrar, somar, subtrair, multiplicar — um laço só
Médio	41 a 65	decisões dentro do laço, dois vetores, busca, extremos
Avançado	66 a 85	ordenação, remoção, frequência, conjuntos
Difícil	86 a 100	algoritmos clássicos, uma passada só, sem vetor auxiliar

Como estudar

1. Resolva na ordem. Os exercícios foram encadeados: o 47 supõe que você já fez o 41.
2. **Tente antes de olhar o gabarito.** Um exercício que você olhou a resposta sem tentar não ensinou nada.
3. Compile e rode de verdade: `gcc exercicio.c -o exercicio -Wall`.
4. Depois de acertar, mude o enunciado sozinho — troque `<` por `<=`, inverta o sentido do laço, mude o tamanho. Ver o que quebra é o que fixa o conteúdo.

O gabarito, na segunda metade do caderno, mostra o **trecho essencial** de cada resposta, e não o programa inteiro. Todos os trechos supõem este esqueleto:

```

1 #include <stdio.h>
2 #define N 10                      /* ajuste o tamanho conforme o exercicio */
3
4 int main(void) {
5     int v[N];
6
7     /* leitura padrao usada em quase todos os exercicios */
8     for (int i = 0; i < N; i++) {
9         printf("v[%d] = ", i);
10        scanf("%d", &v[i]);
11    }
12
13    /* ---- o trecho do gabarito entra aqui ---- */
14
15    return 0;
16 }
```

Regras que valem para os 100 exercícios

- Índices vão de 0 a N-1. Nunca acesse `v[N]`.
- Quando o enunciado disser "posição", entenda **índice** do vetor (começando em 0), a menos que o texto diga "posição 1" explicitamente.
- Quando disser "elementos pares", refere-se ao **valor**; quando disser "posições pares", refere-se ao **índice**.

- Nenhum exercício precisa de `malloc`, matriz, `struct` ou biblioteca além de `<stdio.h>` (exceto onde o gabarito indicar).

1 Nível 1 — Fácil (1 a 40)

O que este nível cobra

Um vetor, um laço, uma conta. Aqui você só precisa de leitura, impressão e as quatro operações básicas. Se um exercício deste nível levar mais de dez minutos, revise a declaração e o percurso de vetores antes de continuar.

1 Leitura e exibição

Leia 5 números inteiros em um vetor e mostre-os na mesma ordem em que foram digitados.

2 Ordem inversa

Leia 10 números inteiros e mostre-os do último para o primeiro.

3 Soma de todos

Leia 10 números e mostre a soma de todos eles.

4 Média simples

Leia 10 números e mostre a média aritmética deles, com duas casas decimais.

5 Somar uma constante

Leia 8 números e mostre cada um deles somado a 3.

6 Subtrair uma constante

Leia 8 números e mostre cada um deles subtraído de 2.

7 Dobrar cada elemento

Leia 10 números e mostre cada um multiplicado por 2.

8 Multiplicar por um valor lido

Leia 6 números e, em seguida, leia um multiplicador m . Mostre cada elemento do vetor multiplicado por m .

9 Soma dos cinco primeiros

Leia 10 números e mostre a soma apenas dos 5 primeiros.

10 Produto de todos

Leia 5 números e mostre o produto (a multiplicação) de todos eles.

11 Vetor do dobro

Leia 8 números em um vetor a . Crie um vetor b em que cada posição contenha o dobro da posição correspondente de a . Mostre os dois vetores.

12 Vetor dos quadrados

Leia 6 números e crie outro vetor com o quadrado de cada um ($v[i] \times v[i]$). Mostre o resultado.

13 Soma de dois vetores

Leia dois vetores a e b de 5 posições cada. Crie um vetor c em que $c[i] = a[i] + b[i]$ e mostre-o.

14 Subtração de dois vetores

Com os mesmos dois vetores do exercício anterior, gere um vetor c em que $c[i] = a[i] - b[i]$.

15 Multiplicação de dois vetores

Gere um vetor `c` em que cada posição seja o produto das posições correspondentes de `a` e `b`.

16 Preenchimento automático de 1 a 10

Sem ler nada do teclado, preencha um vetor de 10 posições com os números de 1 a 10 e mostre-o.

17 Números pares automáticos

Preencha um vetor de 10 posições com os 10 primeiros números pares positivos (2, 4, 6, ..., 20) e mostre-o.

18 Múltiplos de 5

Preencha um vetor de 8 posições com os múltiplos de 5 a partir do 5 (5, 10, 15, ...) e mostre-o.

19 Tabuada em vetor

Leia um número `n` e preencha um vetor de 10 posições com a tabuada de `n` ($n \times 1$ até $n \times 10$). Mostre no formato `n x 1 = resultado`.

20 Contar positivos

Leia 10 números inteiros e informe quantos deles são positivos (maiores que zero).

21 Contar negativos

Com os mesmos 10 números, informe quantos são negativos.

22 Contar zeros

Com os mesmos 10 números, informe quantos são iguais a zero.

23 Somar apenas os positivos

Leia 10 números e mostre a soma somente dos valores positivos.

24 Somar apenas os valores pares

Leia 10 números e mostre a soma dos que são pares (divisíveis por 2).

25 Somar as posições pares

Leia 10 números e mostre a soma dos elementos que estão nos índices pares (0, 2, 4, 6, 8). Atenção: aqui não importa o valor, e sim a posição.

26 Somar as posições ímpares

Com o mesmo vetor, mostre a soma dos elementos que estão nos índices ímpares (1, 3, 5, 7, 9).

27 Metade de cada elemento

Leia 6 números inteiros e mostre a metade de cada um, com uma casa decimal.

28 Índice e valor

Leia 8 números e mostre, em cada linha, o índice e o valor correspondente no formato `Posicao 0: valor` 15.

29 Primeiro mais último

Leia 10 números e mostre a soma do primeiro elemento com o último.

30 Trocar as pontas

Leia 7 números, troque o primeiro elemento com o último e mostre o vetor resultante.

31 Ajuste nas pontas

Leia 6 números, some 10 ao primeiro elemento, subtraia 10 do último, e mostre o vetor.

32 Contando o tamanho

Declare um vetor já inicializado com os valores {4, 8, 15, 16, 23, 42} e descubra, usando `sizeof`, quantos elementos ele tem. Mostre a quantidade e a soma.

33 Copiar vetor

Leia 8 números em um vetor `a` e copie todos eles para um vetor `b`, elemento a elemento. Mostre os dois.

34 Zerar vetor

Leia 6 números, mostre-os, depois zere todas as posições do vetor e mostre novamente.

35 Preencher com valor lido

Leia um número x e preencha todas as 10 posições de um vetor com esse mesmo valor. Mostre o vetor.

36 Incrementar todos

Leia 8 números e some 1 a cada elemento *dentro do próprio vetor* (sem criar outro vetor). Mostre o resultado.

37 Soma acumulada

Leia 6 números e mostre, a cada elemento lido, a soma acumulada até ali. Exemplo: para 2, 5, 3 a saída seria 2, 7, 10.

38 Valor vezes índice

Leia 8 números e mostre cada elemento multiplicado pelo seu próprio índice.

39 Dobrar no lugar

Leia 10 números e substitua cada elemento pelo dobro dele mesmo, alterando o próprio vetor. Mostre antes e depois.

40 Média dos três primeiros

Leia 10 números e mostre a média apenas dos três primeiros elementos, com duas casas decimais.

2 Nível 2 — Médio (41 a 65)

O que este nível cobra

Agora o laço precisa *decidir*. Aparecem condições dentro do percurso, variáveis que guardam estado entre as voltas, dois vetores trabalhando juntos e o primeiro contato com busca. Continua sendo um laço por vez — mas o que acontece dentro dele importa.

41 Maior elemento

Leia 10 números e mostre o maior deles. Não use vetor auxiliar nem ordenação.

42 Menor elemento

Leia 10 números e mostre o menor deles.

43 Maior com posição

Leia 10 números e mostre o maior valor e o índice em que ele está.

44 Menor com posição

Leia 10 números e mostre o menor valor e a posição dele. Se houver empate, mostre a primeira ocorrência.

45 Busca de um valor

Leia 10 números e depois um valor x . Informe se x está no vetor e, em caso positivo, em qual índice. Se não estiver, mostre uma mensagem apropriada.

46 Contar ocorrências

Leia 10 números e um valor x . Informe quantas vezes x aparece no vetor.

47 Quantos acima da média

Leia 10 números, calcule a média e informe quantos elementos estão acima dela.

48 Listar os acima da média

Além da quantidade, mostre *quais* são os elementos acima da média e em que posições estão.

49 Inverter no lugar

Leia 10 números e inverta a ordem dos elementos **no próprio vetor**, sem usar um vetor auxiliar. Use apenas $n/2$ trocas.

50 Cópia invertida

Leia 8 números em **a** e preencha **b** com os mesmos valores em ordem inversa, mantendo **a** intacto.

51 Separar pares e ímpares

Leia 10 números e distribua-os em dois vetores: um só com os valores pares e outro só com os ímpares. Mostre os dois, informando quantos há em cada.

52 Somas separadas

Leia 10 números e mostre, separadamente, a soma dos valores pares e a soma dos valores ímpares.

53 Múltiplos de 3

Leia 12 números e informe quantos são múltiplos de 3 e qual a soma deles.

54 Zerar negativos

Leia 10 números e substitua por zero todos os valores negativos, dentro do próprio vetor. Mostre antes e depois.

55 Trocar o sinal

Leia 8 números e inverta o sinal de todos (positivos viram negativos e vice-versa), no próprio vetor.

56 Amplitude

Leia 10 números e mostre a amplitude do conjunto, isto é, a diferença entre o maior e o menor valor.

57 Soma em um intervalo

Leia 10 números e depois duas posições, i e j . Mostre a soma dos elementos entre essas duas posições (incluindo as duas). Valide se as posições são válidas.

58 Está ordenado?

Leia 10 números e informe se o vetor está em ordem crescente. Não ordene — apenas verifique.

59 Média dos positivos

Leia 10 números e mostre a média apenas dos valores positivos. Se não houver nenhum, avise.

60 Produto escalar

Leia dois vetores de 5 posições e calcule o produto escalar entre eles: $a[0] \cdot b[0] + a[1] \cdot b[1] + \dots$

61 Vetores iguais?

Leia dois vetores de 6 posições e informe se eles são idênticos (mesmos valores nas mesmas posições).

62 Concatenar

Leia um vetor **a** de 5 posições e um **b** de 4. Crie um vetor **c** contendo primeiro todos os de **a** e depois todos os de **b**.

63 Intercalar

Leia dois vetores de 5 posições e crie um terceiro alternando um elemento de cada: $a[0], b[0], a[1], b[1], \dots$

64 Elementos em comum

Leia dois vetores de 6 posições e informe quantos elementos de **a** também aparecem em **b**, mostrando quais são.

65 Deslocar à esquerda

Leia 8 números e desloque todos uma posição para a esquerda: o primeiro elemento vai para o fim, e todos os outros andam um lugar. Faça isso no próprio vetor.

3 Nível 3 — Avançado (66 a 85)

O que este nível cobra

Aqui entram os algoritmos clássicos de vetor: ordenação, remoção com deslocamento, contagem por frequência e operações de conjunto. Vários exercícios exigem dois laços aninhados ou dois índices caminhando em ritmos diferentes. Faça um teste de mesa antes de compilar.

66 Bubble sort

Leia 10 números e ordene-os em ordem crescente usando o método da bolha (compara vizinhos e troca). Inclua a otimização que interrompe o processo quando não houver mais trocas em uma passada.

67 Selection sort

Ordene um vetor de 10 números pelo método da seleção: a cada passo, encontre o menor do trecho restante e traga-o para a posição correta.

68 Insertion sort

Ordene um vetor de 10 números pelo método da inserção, empurrando os maiores para a direita até encontrar o lugar da chave.

69 Busca binária

Leia 10 números **já em ordem crescente** e um valor x . Encontre a posição de x usando busca binária. Mostre também quantas comparações foram necessárias.

70 Inserção mantendo a ordem

Dado um vetor de 9 posições ordenado com 8 elementos em uso, leia um novo valor e insira-o na posição correta, deslocando os maiores para a direita.

71 Remover por posição

Leia 10 números e uma posição p . Remova o elemento dessa posição, deslocando os seguintes para a esquerda, e mostre o vetor com o novo tamanho.

72 Remover todas as ocorrências

Leia 12 números e um valor x . Remova *todas* as ocorrências de x do vetor, mantendo a ordem dos demais. Informe quantas foram removidas.

73 Remover duplicatas

Leia 12 números e gere um vetor com os mesmos valores, porém sem repetições, preservando a ordem da primeira aparição.

74 Tabela de frequências

Leia 20 notas inteiras de 0 a 10 e mostre uma tabela com quantas vezes cada nota apareceu, desenhando um histograma com asteriscos.

75 Moda

Leia 15 números inteiros entre 0 e 20 e informe qual valor aparece mais vezes e quantas vezes ele aparece. Em caso de empate, mostre o menor.

76 Mediana

Leia 11 números, ordene-os e mostre a mediana (o valor central). Faça também o caso de quantidade par, em que a mediana é a média dos dois centrais.

77 Segundo maior

Leia 10 números e encontre o segundo maior valor **sem ordenar** o vetor. Cuidado com valores repetidos.

78 Rotação de k posições

Leia 8 números e um valor k . Rotacione o vetor k posições para a direita: o que sai pelo fim volta pelo começo.

79 União sem repetição

Leia dois vetores de 6 posições e gere um terceiro contendo todos os valores que aparecem em pelo menos um deles, sem repetições.

80 Interseção

Com os mesmos dois vetores, gere um vetor contendo apenas os valores que aparecem nos **dois**, sem repetições.

81 Diferença de conjuntos

Gere um vetor com os valores que aparecem em **a** mas **não** aparecem em **b**.

82 Vetor palíndromo

Leia 9 números e informe se o vetor é um palíndromo, ou seja, se lido de trás para frente é igual ao original.

83 Contar primos

Leia 10 números inteiros positivos e informe quantos deles são primos, mostrando quais são.

84 Mesclar ordenados

Leia dois vetores de 5 posições, cada um já em ordem crescente, e gere um terceiro vetor de 10 posições também ordenado, sem usar nenhum método de ordenação — apenas percorrendo os dois ao mesmo tempo.

85 Pares com soma x

Leia 10 números e um valor x . Informe quantos pares de elementos distintos somam exatamente x , mostrando cada par encontrado uma única vez.

4 Nível 4 — Difícil (86 a 100)

O que este nível cobra

Os quinze finais são problemas clássicos de entrevista técnica e de olimpíadas de programação, todos resolvíveis apenas com vetores. Vários pedem solução em **uma única passada** ou **sem vetor auxiliar** — restrições que existem para forçar a solução elegante em vez da força bruta. Se travar, resolva primeiro pela força bruta e depois procure a otimização.

86 Maior soma de subvetor contíguo

Leia 10 números (podem ser negativos) e encontre a maior soma possível de uma sequência de elementos *consecutivos*. Exemplo: em $\{-2, 1, -3, 4, -1, 2, 1, -5, 4\}$ a resposta é 6, do subvetor $\{4, -1, 2, 1\}$. Resolva em uma única passada.

87 Produto de todos, exceto o próprio

Leia 6 números e gere um vetor em que cada posição contenha o produto de todos os outros elementos, **sem usar divisão**. Exemplo: $\{1, 2, 3, 4\} \rightarrow \{24, 12, 8, 6\}$.

88 Elemento majoritário

Leia 15 números e descubra se existe um valor que aparece **mais da metade** das vezes. Tente resolver em uma passada e sem vetor de contagem (algoritmo de Boyer-Moore).

89 Mover zeros para o fim

Leia 12 números e mova todos os zeros para o final do vetor, **mantendo a ordem relativa** dos demais elementos. Não use vetor auxiliar.

90 Número que falta

Um vetor contém $n - 1$ números distintos entre 1 e n , em qualquer ordem. Descubra qual número está faltando, sem ordenar e sem vetor auxiliar.

91 O único que não se repete

Em um vetor de 11 números, todos aparecem exatamente duas vezes, exceto um. Encontre esse elemento único em uma passada, usando espaço constante. Dica: o operador XOR .

92 Maior sequência crescente

Leia 12 números e informe o comprimento da maior sequência de elementos *consecutivos* em ordem estritamente crescente, mostrando onde ela começa.

93 Alternar positivos e negativos

Leia 10 números contendo positivos e negativos e reorganize o vetor de modo que os sinais se alternem, começando por um positivo. Se sobrares elementos de um sinal, coloque-os no fim.

94 Dois que somam x (vetor ordenado)

Dado um vetor de 10 números **já ordenado** e um valor x , encontre um par de elementos cuja soma seja exatamente x , usando dois índices que caminham das pontas para o meio. Complexidade $O(n)$ — não use dois laços aninhados.

95 Ordenar 0s, 1s e 2s

Leia 12 números contendo apenas os valores 0, 1 e 2 e ordene o vetor em uma **única passada**, sem contar as ocorrências e sem usar método de ordenação (problema da bandeira holandesa).

96 k -ésimo maior

Leia 10 números e um valor k . Encontre o k -ésimo maior elemento do vetor. Resolva sem ordenar o vetor inteiro.

97 Contar inversões

Leia 10 números e conte quantos pares (i, j) existem com $i < j$ e $v[i] > v[j]$. Esse número mede o quão longe o vetor está de estar ordenado.

98 Menor janela a ordenar

Leia 12 números e descubra o menor intervalo contíguo que, se fosse ordenado, deixaria o vetor inteiro ordenado. Mostre os índices inicial e final desse intervalo.

99 Rotação sem vetor auxiliar

Rotacione um vetor de 9 elementos em k posições à esquerda usando apenas três inversões de trechos e nenhuma memória extra além de uma variável temporária.

100 Ponto de equilíbrio

Leia 10 números e encontre um índice em que a soma de todos os elementos à esquerda seja igual à soma de todos os elementos à direita. Se não existir, informe. Resolva em $O(n)$.

5 Gabarito — Nível 1 (1 a 40)

Os trechos abaixo supõem o esqueleto mostrado na abertura do caderno: um vetor `v` de `N` posições já declarado e lido.

1 Leitura e exibição

```
1 int v[5];
2 for (int i = 0; i < 5; i++) { printf("v[%d] = ", i); scanf("%d", &v[i]); }
3 for (int i = 0; i < 5; i++) printf("%d ", v[i]);
4 printf("\n");
```

2 Ordem inversa

```
1 for (int i = 9; i >= 0; i--) printf("%d ", v[i]); // começa no ultimo indice
```

O laço reverso começa em `N-1` e testa `i >= 0`. Usar `i > 0` deixaria de imprimir `v[0]`.

3 Soma de todos

```
1 int soma = 0;
2 for (int i = 0; i < 10; i++) soma += v[i];
3 printf("Soma = %d\n", soma);
```

4 Média simples

```
1 int soma = 0;
2 for (int i = 0; i < 10; i++) soma += v[i];
3 printf("Media = %.2f\n", (float) soma / 10); // o cast e obrigatorio
```

Sem o `(float)`, C faria divisão inteira e a média de 7 e 8 daria 7, não 7.50.

5 Somar uma constante

```
1 for (int i = 0; i < 8; i++) printf("%d ", v[i] + 3);
```

6 Subtrair uma constante

```
1 for (int i = 0; i < 8; i++) printf("%d ", v[i] - 2);
```

7 Dobrar cada elemento

```
1 for (int i = 0; i < 10; i++) printf("%d ", v[i] * 2);
```

8 Multiplicar por um valor lido

```
1 int m;
2 printf("Multiplicador: "); scanf("%d", &m);
3 for (int i = 0; i < 6; i++) printf("%d ", v[i] * m);
```

9 Soma dos cinco primeiros

```
1 int soma = 0;
2 for (int i = 0; i < 5; i++) soma += v[i]; // para em 5, nao em 10
```

10 Produto de todos

```
1 long long produto = 1;           // começa em 1: elemento neutro da multiplicacao
2 for (int i = 0; i < 5; i++) produto *= v[i];
3 printf("Produto = %lld\n", produto);
```

Começar em 0 zeraria tudo. O tipo `long long` evita estouro com valores grandes.

11 Vetor do dobro

```
1 int b[8];
2 for (int i = 0; i < 8; i++) b[i] = a[i] * 2;
```

12 Vetor dos quadrados

```
1 int q[6];
2 for (int i = 0; i < 6; i++) q[i] = v[i] * v[i];
```

13 Soma de dois vetores

```
1 int c[5];
2 for (int i = 0; i < 5; i++) c[i] = a[i] + b[i]; // um indice serve para os tres
```

14 Subtração de dois vetores

```
1 for (int i = 0; i < 5; i++) c[i] = a[i] - b[i];
```

15 Multiplicação de dois vetores

```
1 for (int i = 0; i < 5; i++) c[i] = a[i] * b[i];
```

16 Preenchimento automático de 1 a 10

```
1 int v[10];
2 for (int i = 0; i < 10; i++) v[i] = i + 1; // indice 0 guarda o valor 1
```

17 Números pares automáticos

```
1 for (int i = 0; i < 10; i++) v[i] = (i + 1) * 2; // 2, 4, 6, ..., 20
```

18 Múltiplos de 5

```
1 for (int i = 0; i < 8; i++) v[i] = (i + 1) * 5;
```

19 Tabuada em vetor

```
1 int n, t[10];
2 scanf("%d", &n);
3 for (int i = 0; i < 10; i++) t[i] = n * (i + 1);
4 for (int i = 0; i < 10; i++) printf("%d x %d = %d\n", n, i + 1, t[i]);
```

20 Contar positivos

```
1 int cont = 0;
2 for (int i = 0; i < 10; i++) if (v[i] > 0) cont++;
```

21 Contar negativos

```
1 int cont = 0;
2 for (int i = 0; i < 10; i++) if (v[i] < 0) cont++;
```

22 Contar zeros

```
1 int cont = 0;
2 for (int i = 0; i < 10; i++) if (v[i] == 0) cont++;
```

Os três contadores acima podem conviver em um único laço com `if / else if / else`.

23 Somar apenas os positivos

```
1 int soma = 0;
2 for (int i = 0; i < 10; i++) if (v[i] > 0) soma += v[i];
```

24 Somar apenas os valores pares

```
1 int soma = 0;
2 for (int i = 0; i < 10; i++) if (v[i] % 2 == 0) soma += v[i];
```

25 Somar as posições pares

```
1 int soma = 0;
2 for (int i = 0; i < 10; i += 2) soma += v[i]; // o PASSO cuida das posicoes
```

Repare na diferença para o exercício anterior: aqui a seleção está no *passo do laço*, e não em um `if` sobre o valor.

26 Somar as posições ímpares

```
1 int soma = 0;
2 for (int i = 1; i < 10; i += 2) soma += v[i]; // começa em 1
```

27 Metade de cada elemento

```
1 for (int i = 0; i < 6; i++) printf("%.1f ", v[i] / 2.0); // 2.0 força float
```

28 Índice e valor

```
1 for (int i = 0; i < 8; i++) printf("Posicao %d: valor %d\n", i, v[i]);
```

29 Primeiro mais último

```
1 printf("%d\n", v[0] + v[9]); // v[9] é o ultimo de um vetor de 10
```

30 Trocar as pontas

```
1 int tmp = v[0]; // a variavel temporaria é obrigatoria
2 v[0] = v[6];
3 v[6] = tmp;
```

Sem o `tmp`, a primeira atribuição destruiria `v[0]` antes de ele ser copiado.

31 Ajuste nas pontas

```
1 v[0] += 10;
2 v[5] -= 10;
```

32 Contando o tamanho

```
1 int v[] = {4, 8, 15, 16, 23, 42};
2 int n = sizeof v / sizeof v[0];      // 24 / 4 = 6
3 int soma = 0;
4 for (int i = 0; i < n; i++) soma += v[i];
5 printf("%d elementos, soma %d\n", n, soma);
```

33 Copiar vetor

```
1 int b[8];
2 for (int i = 0; i < 8; i++) b[i] = a[i]; // b = a NAO funciona em C
```

34 Zerar vetor

```
1 for (int i = 0; i < 6; i++) v[i] = 0;
```

35 Preencher com valor lido

```
1 int x;
2 scanf("%d", &x);
3 for (int i = 0; i < 10; i++) v[i] = x;
```

36 Incrementar todos

```
1 for (int i = 0; i < 8; i++) v[i]++; // altera o proprio vetor
```

37 Soma acumulada

```
1 int soma = 0;
2 for (int i = 0; i < 6; i++) {
3     soma += v[i];
4     printf("%d ", soma); // imprime DENTRO do laço
5 }
```

38 Valor vezes índice

```
1 for (int i = 0; i < 8; i++) printf("%d ", v[i] * i);
```

39 Dobrar no lugar

```
1 for (int i = 0; i < 10; i++) printf("%d ", v[i]); // antes
2 for (int i = 0; i < 10; i++) v[i] *= 2;
3 for (int i = 0; i < 10; i++) printf("%d ", v[i]); // depois
```

40 Média dos três primeiros

```
1 printf("%.2f\n", (v[0] + v[1] + v[2]) / 3.0);
```

6 Gabarito — Nível 2 (41 a 65)

41 Maior elemento

```
1 int maior = v[0];           // NUNCA comece com 0
2 for (int i = 1; i < 10; i++) if (v[i] > maior) maior = v[i];
```

Iniciar com 0 daria resposta errada se todos os valores fossem negativos.

42 Menor elemento

```
1 int menor = v[0];
2 for (int i = 1; i < 10; i++) if (v[i] < menor) menor = v[i];
```

43 Maior com posição

```
1 int pos = 0;
2 for (int i = 1; i < 10; i++) if (v[i] > v[pos]) pos = i;
3 printf("Maior = %d na posicao %d\n", v[pos], pos);
```

Guardar o *índice* em vez do valor é mais econômico: o valor sempre pode ser obtido com `v[pos]`.

44 Menor com posição

```
1 int pos = 0;
2 for (int i = 1; i < 10; i++) if (v[i] < v[pos]) pos = i; // "<" mantém o 1o em empate
```

45 Busca de um valor

```
1 int x, achou = -1;
2 scanf("%d", &x);
3 for (int i = 0; i < 10; i++)
4     if (v[i] == x) { achou = i; break; } // para na 1a ocorrência
5 printf(achou >= 0 ? "Achado na posicao %d\n" : "Nao encontrado\n", achou);
```

46 Contar ocorrências

```
1 int x, cont = 0;
2 scanf("%d", &x);
3 for (int i = 0; i < 10; i++) if (v[i] == x) cont++; // SEM break: conta todas
```

47 Quantos acima da média

```
1 int soma = 0;
2 for (int i = 0; i < 10; i++) soma += v[i];
3 float media = (float) soma / 10;
4
5 int cont = 0; // SEGUNDO laço: a media
6 for (int i = 0; i < 10; i++) // so existe depois do primeiro
7     if (v[i] > media) cont++;
```

48 Listar os acima da média

```
1 for (int i = 0; i < 10; i++)
2     if (v[i] > media) printf("v[%d] = %d\n", i, v[i]);
```

49 Inverter no lugar

```
1 for (int i = 0, j = 9; i < j; i++, j--) { // dois indices, sentidos opostos
2     int t = v[i]; v[i] = v[j]; v[j] = t;
3 }
```

A condição $i < j$ é a chave: quando os índices se cruzam, o trabalho acabou. Percorrer até $i < n$ inverteria o vetor duas vezes, devolvendo o original.

50 Cópia invertida

```
1 for (int i = 0; i < 8; i++) b[i] = a[7 - i]; // ou b[7-i] = a[i]
```

51 Separar pares e ímpares

```
1 int par[10], impar[10], np = 0, ni = 0;
2 for (int i = 0; i < 10; i++) {
3     if (v[i] % 2 == 0) par[np++] = v[i];
4     else impar[ni++] = v[i];
5 }
```

Cada vetor de destino tem seu **próprio contador**, que só avança quando algo é gravado nele.

52 Somas separadas

```
1 int somaP = 0, somaI = 0;
2 for (int i = 0; i < 10; i++) {
3     if (v[i] % 2 == 0) somaP += v[i];
4     else somaI += v[i];
5 }
```

53 Múltiplos de 3

```
1 int cont = 0, soma = 0;
2 for (int i = 0; i < 12; i++)
3     if (v[i] % 3 == 0) { cont++; soma += v[i]; }
```

54 Zerar negativos

```
1 for (int i = 0; i < 10; i++) if (v[i] < 0) v[i] = 0;
```

55 Trocar o sinal

```
1 for (int i = 0; i < 8; i++) v[i] = -v[i];
```

56 Amplitude

```
1 int maior = v[0], menor = v[0];
2 for (int i = 1; i < 10; i++) {
3     if (v[i] > maior) maior = v[i];
4     if (v[i] < menor) menor = v[i]; // os dois no MESMO laço
5 }
6 printf("Amplitude = %d\n", maior - menor);
```

57 Soma em um intervalo

```
1 int a, b;
2 scanf("%d %d", &a, &b);
3 if (a > b) { int t = a; a = b; b = t; } // aceita ordem invertida
4 if (a < 0 || b > 9) { printf("Intervalo invalido\n"); return 1; }
5
6 int soma = 0;
7 for (int i = a; i <= b; i++) soma += v[i]; // <= inclui o extremo
```

58 Está ordenado?

```
1 int ordenado = 1;
2 for (int i = 0; i < 9; i++) // ate n-1: usa v[i+1]
3     if (v[i] > v[i + 1]) { ordenado = 0; break; }
```

Sempre que o corpo usar `v[i+1]`, o laço precisa parar em `n-1`.

59 Média dos positivos

```
1 int soma = 0, cont = 0;
2 for (int i = 0; i < 10; i++)
3     if (v[i] > 0) { soma += v[i]; cont++; }
4
5 if (cont == 0) printf("Nenhum positivo\n"); // evita divisao por zero
6 else         printf("%.2f\n", (float) soma / cont);
```

60 Produto escalar

```
1 int prod = 0;
2 for (int i = 0; i < 5; i++) prod += a[i] * b[i];
```

61 Vetores iguais?

```
1 int iguais = 1;
2 for (int i = 0; i < 6; i++)
3     if (a[i] != b[i]) { iguais = 0; break; }
```

62 Concatenar

```
1 int c[9], k = 0;
2 for (int i = 0; i < 5; i++) c[k++] = a[i];
3 for (int i = 0; i < 4; i++) c[k++] = b[i]; // k continua de onde parou
```

63 Intercalar

```
1 int c[10], k = 0;
2 for (int i = 0; i < 5; i++) {
3     c[k++] = a[i];
4     c[k++] = b[i];
5 }
```

64 Elementos em comum

```
1 int cont = 0;
2 for (int i = 0; i < 6; i++)
3     for (int j = 0; j < 6; j++)
4         if (a[i] == b[j]) {
5             printf("%d ", a[i]);
6             cont++;
7             break; // evita contar duas vezes o mesmo a[i]
8         }
```

65 Deslocar à esquerda

```

1 int primeiro = v[0];           // guarda antes de perder
2 for (int i = 0; i < 7; i++) v[i] = v[i + 1]; // todos andam um lugar
3 v[7] = primeiro;               // o primeiro volta pelo fim

```

Aqui o laço vai *da frente para trás*, porque estamos puxando valores para a esquerda. Se fosse empurrar para a direita, o sentido teria que ser invertido.

7 Gabarito — Nível 3 (66 a 85)**66 Bubble sort**

```

1 for (int i = 0; i < 9; i++) {
2     int trocou = 0;
3     for (int j = 0; j < 9 - i; j++) {           // "-i": o fim ja esta ordenado
4         if (v[j] > v[j + 1]) {
5             int t = v[j]; v[j] = v[j+1]; v[j+1] = t;
6             trocou = 1;
7         }
8     }
9     if (!trocou) break;                         // ja ordenado: para cedo
10 }

```

67 Selection sort

```

1 for (int i = 0; i < 9; i++) {
2     int menor = i;
3     for (int j = i + 1; j < 10; j++)
4         if (v[j] < v[menor]) menor = j;
5     if (menor != i) { int t=v[i]; v[i]=v[menor]; v[menor]=t; }
6 }

```

Faz no máximo $n - 1$ trocas — o menor número entre os métodos simples.

68 Insertion sort

```

1 for (int i = 1; i < 10; i++) {
2     int chave = v[i], j = i - 1;
3     while (j >= 0 && v[j] > chave) {           // empurra os maiores
4         v[j + 1] = v[j];
5         j--;
6     }
7     v[j + 1] = chave;                         // encaixa
8 }

```

69 Busca binária

```

1 int x, ini = 0, fim = 9, pos = -1, passos = 0;
2 scanf("%d", &x);
3 while (ini <= fim) {
4     int meio = ini + (fim - ini) / 2;         // evita overflow de (ini+fim)
5     passos++;
6     if (v[meio] == x) { pos = meio; break; }
7     if (v[meio] < x) ini = meio + 1;
8     else fim = meio - 1;
9 }
10 printf("pos=%d em %d passos\n", pos, passos);

```

70 Inserção mantendo a ordem

```

1  int n = 8, x;
2  scanf("%d", &x);
3
4  int pos = n;                                // supoe que vai para o fim
5  for (int i = 0; i < n; i++)
6      if (v[i] > x) { pos = i; break; }        // achou o lugar
7
8  for (int i = n; i > pos; i--) v[i] = v[i-1]; // abre espaco: DE TRAS PARA FRENTE
9  v[pos] = x;
10 n++;

```

71 Remover por posição

```

1  int n = 10, p;
2  scanf("%d", &p);
3  if (p >= 0 && p < n) {
4      for (int i = p; i < n - 1; i++) v[i] = v[i + 1]; // fecha o buraco
5      n--;
6  }

```

Abrir espaço percorre de trás para frente; fechar buraco, da frente para trás. Trocar o sentido corrompe os dados.

72 Remover todas as ocorrências

```

1  int x, n = 12, k = 0;
2  scanf("%d", &x);
3  for (int i = 0; i < n; i++)
4      if (v[i] != x) v[k++] = v[i];           // regravando so o que fica
5  printf("Removidos: %d\n", n - k);
6  n = k;

```

Truque dos dois índices: *i* lê sempre, *k* só avança quando grava. Resolve em uma passada, sem vetor auxiliar.

73 Remover duplicatas

```

1  int saida[12], m = 0;
2  for (int i = 0; i < 12; i++) {
3      int repetido = 0;
4      for (int j = 0; j < m; j++)                // procura no que JA saiu
5          if (saida[j] == v[i]) { repetido = 1; break; }
6      if (!repetido) saida[m++] = v[i];
7  }

```

74 Tabela de frequências

```

1  int freq[11] = {0};                        // indices 0..10
2  for (int i = 0; i < 20; i++) freq[v[i]]++; // o VALOR vira INDICE
3
4  for (int k = 0; k <= 10; k++) {
5      printf("%2d | %2d | ", k, freq[k]);
6      for (int b = 0; b < freq[k]; b++) putchar('#');
7      putchar('\n');
8  }

```

`freq[v[i]]++` só funciona porque os valores estão garantidamente entre 0 e 10 — valide a entrada, ou será um estouro de vetor.

75 Moda

```

1 int freq[21] = {0};
2 for (int i = 0; i < 15; i++) freq[v[i]]++;
3
4 int moda = 0;
5 for (int k = 1; k <= 20; k++)
6     if (freq[k] > freq[moda]) moda = k;          // ">" mantém o MENOR em empate
7 printf("Moda = %d (%d vezes)\n", moda, freq[moda]);

```

76 Mediana

```

1 /* 1. ordena (qualquer metodo) ... */
2 /* 2. pega o centro */
3 if (n % 2 == 1) printf("%d\n", v[n/2]);
4 else           printf("%.1f\n", (v[n/2 - 1] + v[n/2]) / 2.0);

```

77 Segundo maior

```

1 int m1 = v[0], m2 = -2147483647;          // m2 começa "muito pequeno"
2 for (int i = 1; i < 10; i++) {
3     if (v[i] > m1)                        { m2 = m1; m1 = v[i]; } // novo campeão
4     else if (v[i] > m2 && v[i] < m1) m2 = v[i];          // novo vice
5 }

```

A ordem importa: o campeão antigo precisa virar vice *antes* de ser substituído. O teste `v[i] < m1` evita que um valor repetido do maior seja considerado o segundo.

78 Rotação de k posições

```

1 int n = 8, k;
2 scanf("%d", &k);
3 k = k % n;                                // rotacionar 10 em 8 = rotacionar 2
4 for (int r = 0; r < k; r++) {
5     int ultimo = v[n-1];
6     for (int i = n - 1; i > 0; i--) v[i] = v[i-1];
7     v[0] = ultimo;
8 }

```

Solução direta, $O(nk)$. O exercício 99 mostra a versão $O(n)$.

79 União sem repetição

```

1 int u[12], m = 0;
2
3 for (int i = 0; i < 6; i++) {              // primeiro os de a
4     int rep = 0;
5     for (int j = 0; j < m; j++) if (u[j] == a[i]) { rep = 1; break; }
6     if (!rep) u[m++] = a[i];
7 }
8 for (int i = 0; i < 6; i++) {              // depois os de b
9     int rep = 0;
10    for (int j = 0; j < m; j++) if (u[j] == b[i]) { rep = 1; break; }
11    if (!rep) u[m++] = b[i];
12 }

```

80 Interseção

```

1 int inter[6], m = 0;
2 for (int i = 0; i < 6; i++) {
3     int estaEmB = 0, jaSaiu = 0;
4     for (int j = 0; j < 6; j++) if (a[i] == b[j]) { estaEmB = 1; break; }
5     for (int j = 0; j < m; j++) if (a[i] == inter[j]) { jaSaiu = 1; break; }
6     if (estaEmB && !jaSaiu) inter[m++] = a[i];
7 }

```

81 Diferença de conjuntos

```

1 int d[6], m = 0;
2 for (int i = 0; i < 6; i++) {
3     int estaEmB = 0;
4     for (int j = 0; j < 6; j++) if (a[i] == b[j]) { estaEmB = 1; break; }
5     if (!estaEmB) d[m++] = a[i];
6 }

```

82 Vetor palíndromo

```

1 int palindromo = 1;
2 for (int i = 0, j = 8; i < j; i++, j--)
3     if (v[i] != v[j]) { palindromo = 0; break; }

```

Mesmo padrão de dois índices da inversão — só que aqui comparamos em vez de trocar.

83 Contar primos

```

1 int ehPrimo(int n) {
2     if (n < 2) return 0;
3     for (int d = 2; d * d <= n; d++) // basta ir ate a raiz de n
4         if (n % d == 0) return 0;
5     return 1;
6 }
7
8 int cont = 0;
9 for (int i = 0; i < 10; i++)
10    if (ehPrimo(v[i])) { printf("%d ", v[i]); cont++; }

```

84 Mesclar ordenados

```

1 int c[10], i = 0, j = 0, k = 0;
2
3 while (i < 5 && j < 5) // enquanto os DOIS tem elementos
4     c[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
5
6 while (i < 5) c[k++] = a[i++]; // sobrou de a
7 while (j < 5) c[k++] = b[j++]; // sobrou de b

```

Este é o *merge* do merge sort. Custa $O(n)$ — muito melhor que concatenar e ordenar.

85 Pares com soma x

```

1 int x, cont = 0;
2 scanf("%d", &x);
3 for (int i = 0; i < 10; i++)
4     for (int j = i + 1; j < 10; j++) // j = i+1: nao repete pares
5         if (v[i] + v[j] == x) {
6             printf("(%d, %d) ", v[i], v[j]);

```

```

7         cont++;
8     }

```

Começar j em $i+1$ evita comparar um elemento com ele mesmo e evita contar o par (a, b) e depois (b, a) .

8 Gabarito — Nível 4 (86 a 100)

86 Maior soma de subvetor contíguo (Kadane)

```

1 int melhor = v[0], atual = v[0];
2 for (int i = 1; i < 10; i++) {
3     atual = (v[i] > atual + v[i]) ? v[i] : atual + v[i];
4     melhor = (atual > melhor) ? atual : melhor;
5 }
6 printf("Maior soma = %d\n", melhor);

```

A pergunta em cada posição é: "vale mais continuar a soma que vinha, ou recomeçar aqui?". Se a soma acumulada ficou negativa, ela só atrapalha — recomeçar é melhor. Uma passada, $O(n)$, sem memória extra. A força bruta com dois laços seria $O(n^2)$.

87 Produto de todos, exceto o próprio

```

1 int n = 6, r[6];
2
3 r[0] = 1;
4 for (int i = 1; i < n; i++) r[i] = r[i-1] * v[i-1]; // produto da ESQUERDA
5
6 int dir = 1;
7 for (int i = n - 1; i >= 0; i--) { // produto da DIREITA
8     r[i] *= dir;
9     dir *= v[i];
10 }

```

Duas passadas: a primeira guarda em $r[i]$ o produto de tudo à esquerda de i ; a segunda multiplica pelo produto de tudo à direita, acumulado em uma única variável. Sem divisão — o que também resolve o caso de haver zeros no vetor.

88 Elemento majoritário (Boyer-Moore)

```

1 int candidato = v[0], votos = 1;
2 for (int i = 1; i < 15; i++) {
3     if (votos == 0) { candidato = v[i]; votos = 1; }
4     else if (v[i] == candidato) votos++;
5     else votos--; // anula um voto contrario
6 }
7
8 int cont = 0; // 2a passada: CONFIRMA
9 for (int i = 0; i < 15; i++) if (v[i] == candidato) cont++;
10 printf(cont > 15/2 ? "Majoritario: %d\n" : "Nao existe\n", candidato);

```

A intuição: cada elemento diferente "cancela" um voto do candidato. Se um valor aparece mais da metade das vezes, é impossível cancelá-lo por completo. A segunda passada é obrigatória, porque o algoritmo sempre devolve *algum* candidato.

89 Mover zeros para o fim

```

1 int n = 12, k = 0;
2 for (int i = 0; i < n; i++)
3     if (v[i] != 0) v[k++] = v[i];      // compacta os nao-zeros
4 while (k < n) v[k++] = 0;             // completa com zeros

```

Mesmo padrão dos dois índices do exercício 72. A ordem relativa é preservada porque a gravação segue a ordem de leitura.

90 Número que falta

```

1 int n = 10;                                // vetor tem n-1 = 9 elementos
2 long esperada = (long) n * (n + 1) / 2;    // soma de 1 ate n
3 long real = 0;
4 for (int i = 0; i < n - 1; i++) real += v[i];
5 printf("Falta o %ld\n", esperada - real);

```

Uma fórmula fechada substitui a busca. Alternativa igualmente elegante: fazer XOR de todos os números de 1 a n com todos os elementos do vetor — o que sobra é o ausente.

91 O único que não se repete

```

1 int unico = 0;
2 for (int i = 0; i < 11; i++) unico ^= v[i]; // XOR de tudo
3 printf("Unico = %d\n", unico);

```

O XOR tem duas propriedades que resolvem o problema: $x \oplus x = 0$ e $x \oplus 0 = x$. Todos os pares se anulam, e sobra exatamente o elemento sem par. $O(n)$ de tempo, $O(1)$ de espaço.

92 Maior sequência crescente

```

1 int melhor = 1, atual = 1, iniMelhor = 0, iniAtual = 0;
2 for (int i = 1; i < 12; i++) {
3     if (v[i] > v[i-1]) atual++;
4     else { atual = 1; iniAtual = i; } // quebrou: recomeça aqui
5     if (atual > melhor) { melhor = atual; iniMelhor = iniAtual; }
6 }
7 printf("Tamanho %d começando em %d\n", melhor, iniMelhor);

```

93 Alternar positivos e negativos

```

1 int pos[10], neg[10], np = 0, nn = 0;
2 for (int i = 0; i < 10; i++)
3     if (v[i] >= 0) pos[np++] = v[i]; else neg[nn++] = v[i];
4
5 int k = 0, i = 0, j = 0;
6 while (i < np && j < nn) { v[k++] = pos[i++]; v[k++] = neg[j++]; }
7 while (i < np) v[k++] = pos[i++]; // sobra de um dos sinais
8 while (j < nn) v[k++] = neg[j++];

```

94 Dois que somam x (dois ponteiros)

```

1 int x, i = 0, j = 9;
2 scanf("%d", &x);
3 while (i < j) {
4     int s = v[i] + v[j];
5     if (s == x) { printf("%d + %d\n", v[i], v[j]); break; }
6     if (s < x) i++; // soma pequena: precisa de um valor MAIOR
7     else j--; // soma grande: precisa de um valor MENOR

```

```
8 }

```

Só funciona porque o vetor está ordenado: cada comparação descarta com segurança um candidato inteiro. $O(n)$ contra $O(n^2)$ da força bruta.

95 Ordenar 0s, 1s e 2s (bandeira holandesa)

```
1 int baixo = 0, meio = 0, alto = 11;
2 while (meio <= alto) {
3     if (v[meio] == 0) {
4         int t=v[baixo]; v[baixo]=v[meio]; v[meio]=t;
5         baixo++; meio++;
6     } else if (v[meio] == 1) {
7         meio++; // ja esta na regioao certa
8     } else { // v[meio] == 2
9         int t=v[alto]; v[alto]=v[meio]; v[meio]=t;
10        alto--; // NAO avanca meio: o novo valor e desconhecido
11    }
12 }

```

Três regiões crescendo ao mesmo tempo: zeros antes de **baixo**, dois depois de **alto**, e uns no meio. Uma única passada, $O(n)$.

96 k -ésimo maior

```
1 int k;
2 scanf("%d", &k);
3 int copia[10];
4 for (int i = 0; i < 10; i++) copia[i] = v[i]; // preserva o original
5
6 for (int i = 0; i < k; i++) { // k passadas de selection sort
7     int maior = i;
8     for (int j = i + 1; j < 10; j++)
9         if (copia[j] > copia[maior]) maior = j;
10    int t=copia[i]; copia[i]=copia[maior]; copia[maior]=t;
11 }
12 printf("%d-esimo maior = %d\n", k, copia[k-1]);

```

Custa $O(nk)$ em vez de $O(n \log n)$ — melhor quando k é pequeno. Note que ordenar *parcialmente* basta: não há razão para ordenar o vetor inteiro.

97 Contar inversões

```
1 int inv = 0;
2 for (int i = 0; i < 9; i++)
3     for (int j = i + 1; j < 10; j++)
4         if (v[i] > v[j]) inv++;
5 printf("Inversoes: %d\n", inv);

```

Zero inversões significa vetor ordenado; $n(n-1)/2$ significa ordem totalmente invertida. Existe solução $O(n \log n)$ adaptando o merge sort, mas a força bruta já responde ao enunciado.

98 Menor janela a ordenar

```
1 int n = 12, ini = -1, fim = -1;
2
3 for (int i = 0; i < n - 1; i++) // 1a quebra da esquerda
4     if (v[i] > v[i+1]) { ini = i; break; }
5
6 if (ini == -1) { printf("Ja ordenado\n"); return 0; }

```

```

7
8 for (int i = n - 1; i > 0; i--) // 1a quebra da direita
9     if (v[i] < v[i-1]) { fim = i; break; }
10
11 int menor = v[ini], maior = v[ini]; // extremos DENTRO da janela
12 for (int i = ini; i <= fim; i++) {
13     if (v[i] < menor) menor = v[i];
14     if (v[i] > maior) maior = v[i];
15 }
16 while (ini > 0 && v[ini-1] > menor) ini--; // expande para a esquerda
17 while (fim < n - 1 && v[fim+1] < maior) fim++; // expande para a direita
18 printf("Janela: [%d, %d]\n", ini, fim);

```

A parte sutil é a expansão: um elemento fora da janela que seja maior que o menor de dentro também está fora de lugar, e precisa entrar.

99 Rotação sem vetor auxiliar (três inversões)

```

1 void inverter(int v[], int i, int j) {
2     while (i < j) { int t=v[i]; v[i]=v[j]; v[j]=t; i++; j--; }
3 }
4
5 int n = 9, k;
6 scanf("%d", &k);
7 k = k % n;
8
9 inverter(v, 0, k - 1); // inverte o primeiro trecho
10 inverter(v, k, n - 1); // inverte o segundo trecho
11 inverter(v, 0, n - 1); // inverte tudo -> rotacao pronta

```

Exemplo com $\{1, 2, 3, 4, 5\}$ e $k = 2$: $\{2, 1, 3, 4, 5\} \rightarrow \{2, 1, 5, 4, 3\} \rightarrow \{3, 4, 5, 1, 2\}$. Custa $O(n)$ e usa apenas uma variável temporária, contra $O(nk)$ da solução do exercício 78.

100 Ponto de equilíbrio

```

1 int total = 0;
2 for (int i = 0; i < 10; i++) total += v[i];
3
4 int esquerda = 0;
5 for (int i = 0; i < 10; i++) {
6     int direita = total - esquerda - v[i]; // deduz em vez de recalculer
7     if (esquerda == direita) {
8         printf("Equilibrio no indice %d\n", i);
9         return 0;
10    }
11    esquerda += v[i];
12 }
13 printf("Nao existe ponto de equilibrio\n");

```

O truque é não recalculer a soma da direita a cada posição (o que daria $O(n^2)$): sabendo o total e a soma da esquerda, a direita sai por subtração. Duas passadas, $O(n)$.

Chegou ao 100?

Se você resolveu os 100 sozinho, domina vetores melhor do que a maioria dos estudantes que concluem a disciplina. O que os quinze últimos ensinam não é sintaxe — é a diferença entre *fazer funcionar* e *fazer bem*: quase todos têm uma solução óbvia $O(n^2)$ e uma solução elegante $O(n)$, e reconhecer a

segunda é o que se espera de um bom programador.

Um último exercício, sem gabarito: escolha dez destes cem, apague seu código, e refaça daqui a uma semana sem consultar nada. O que você reconstruir sozinho é o que realmente aprendeu.